

Conceptos básicos en el procesamiento de texto

Julian D. Echeverry-Correa
jde@utp.edu.co

Técnicas de Procesamiento de Lenguaje Natural

Doctorado en Ingeniería

Maestría en Ingeniería Eléctrica

Maestría en Ingeniería de Sistemas

Maestría en Investigación Operativa y Estadística

Grupo de Investigación en Análisis de Datos y Sociología Computacional - GIADSc
Universidad Tecnológica de Pereira - 2025



- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization
- 4 Procesamiento de texto - Normalización y lematización
- 5 Herramientas

- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization
- 4 Procesamiento de texto - Normalización y lematización
- 5 Herramientas

- Dentro de las **Técnicas de Procesamiento de Lenguaje Natural**, el procesamiento de texto es una etapa común a cualquiera de los campos de aplicación.
- Bien sea en el procesamiento de habla, extracción de información de páginas web, *Sentiment Analysis*, *Opinion Mining*, etc. las etapas de procesamiento básico de información pasan en algún momento por el procesamiento de los textos contenidos en los datos.

- Es importante tener en cuenta que el **Procesamiento de Textos** es dependiente de la tarea. No se debería aplicar el mismo procesamiento en tareas de *Sentiment Analysis* que en tareas de clasificación de textos.
- La razón de esto es simple: existen elementos textuales que aportan información de contexto a cada tarea.

El jugador manifestó en sus redes:
“ES INCREÍBLE LO DE LOS ÁRBITROS.
SIEMPRE PITAN A FAVOR DEL
NACIONAL 🙄😡🔪”

- A continuación veremos algunas de las técnicas más empleadas en el pre-procesamiento básico de datos dentro del **PLN**.

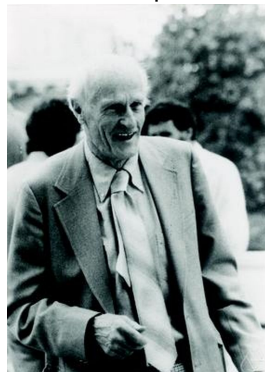
Contenido

- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization
- 4 Procesamiento de texto - Normalización y lematización
- 5 Herramientas

Expresiones regulares

- Las **expresiones regulares** hacen parte de un lenguaje formal para especificar cadenas de textos. Son en sí mismas, cadenas de caracteres con una sintaxis propia.
- Las **expresiones regulares** nos permiten identificar patrones en cadenas de texto.
- ¿En qué las podríamos usar?
 - Encontrar todas las urls en un texto.
 - Identificar las direcciones de correo electrónico.
 - Eliminar o remplazar caracteres indeseados.

La invención de las expresiones regulares se la debemos a Stephen Cole Kleene



- ¿Cómo se pueden formular búsquedas para encontrar las siguientes palabras?

- carro
- Carro
- carros
- Carros



Expresiones regulares

- Caracteres dentro de corchetes.

Patrón	Equivalencias
[cC]arro	carro, Carro
[1234567890]	Cualquier dígito

- Rangos de caracteres

Patrón	Equivalencias	Ejemplos
[A-Z]	Una letra mayúscula	<u>U</u> niversidad Tecnológica
[a-z]	Una letra minúscula	<u>n</u> o quiero ir a clase
[0-9]	Un dígito	Capítulo <u>1</u> : Circuitos básicos

- Negaciones [[^]Ss].
 - Este símbolo ([^]) significa negación sólo cuando va de primero dentro de corchetes.

Patrón	Equivalencia	Ejemplos
[[^] A-Z]	No una mayúscula	<u>C</u> onstitución Política
[[^] Ss]	No 'S' ni 's'	<u>T</u> engo un gusto exquisito
[[^] e [^]]	No e ni [^]	<u>L</u> ook here
a [^] b	El patrón a [^] b	El conjunto <u>a</u> [^] <u>b</u> de elementos

- Otras posibles disyuntivas.

Hola viejo mundo!

Patrón	Ejemplos
Viejo mundo	Hola viejo mundo!
viejo mundo!	Hola <u>viejo</u> <u>mundo</u> !
[Hh]ola [Vv]iejo	<u>Hola</u> <u>viejo</u> mundo!
[Mm]undo	Hola viejo mundo!

- Recordemos que las expresiones regulares son sensibles a las mayúsculas (*case sensitives*)

- Otras posibles disyuntivas.

Hola viejo mundo!

Patrón	Ejemplos
Viejo mundo	Hola viejo mundo!
viejo mundo!	Hola <u>viejo</u> <u>mundo</u> !
[Hh]ola [Vv]iejo	<u>Hola</u> <u>viejo</u> mundo!
[Mm]undo	Hola viejo mundo!

- Recordemos que las expresiones regulares son sensibles a las mayúsculas (*case sensitives*)

- Comodines (*wild cards*).

Patrón	Equivalencia	
colou?r	Caracter previo opcional	<u>color</u> <u>colour</u>
oo*h!	0 o más caracteres previos	<u>oh!</u> <u>ooh!</u> <u>oooh!</u> <u>ooooh!</u>
o+h!	1 o más caracteres previos	<u>oh!</u> <u>ooh!</u> <u>oooh!</u> <u>ooooh!</u>
baa+		<u>baa</u> <u>baaa</u> <u>baaaa</u> <u>baaaaa</u>
beg.n		<u>begin</u> <u>begun</u> <u>begun</u> <u>beg3n</u>

- Caracteres ancla (*anchors*).

Patrón	Equivalencia	
<code>^[A-Z]</code>	<u>P</u> alo Alto	
<code>^[^A-Za-z]</code>	<u>1</u>	" <u>H</u> ola"
<code>\.\$</code>	El fin <u>.</u>	
<code>.\$</code>	El fin <u>?</u>	El fin <u>!</u>

Ejemplo de la necesidad del uso de expresiones regulares

- Supongamos que queremos encontrar todas las instancias de la palabra “**de**” en un texto.
 - **de** No encuentra las instancias con letras mayúsculas
 - **[dD]e** Devuelve incorrectamente palabras como **adrede** o **demonstración**
 - **[^a-zA-Z][dD]e[^a-zA-Z]** Encuentra lo que buscábamos

Ejemplo de la necesidad del uso de expresiones regulares

- Supongamos que queremos encontrar todas las instancias de la palabra “**de**” en un texto.
 - **de** No encuentra las instancias con letras mayúsculas
 - `[dD]e` Devuelve incorrectamente palabras como `adrede` o `demonstración`
 - `[^a-zA-Z][dD]e[^a-zA-Z]` Encuentra lo que buscábamos

Ejemplo de la necesidad del uso de expresiones regulares

- Supongamos que queremos encontrar todas las instancias de la palabra “**de**” en un texto.
 - **de** No encuentra las instancias con letras mayúsculas
 - **[dD]e** Devuelve incorrectamente palabras como **adrede** o **demonstración**
 - **[^a-zA-Z][dD]e[^a-zA-Z]** Encuentra lo que buscábamos

Ejemplo de la necesidad del uso de expresiones regulares

- Supongamos que queremos encontrar todas las instancias de la palabra “**de**” en un texto.
 - **de** No encuentra las instancias con letras mayúsculas
 - **[dD]e** Devuelve incorrectamente palabras como **adrede** o **demonstración**
 - **[^a-zA-Z][dD]e[^a-zA-Z]** Encuentra lo que buscábamos

- El proceso anterior se basó en tratar de arreglar dos tipos de errores
 - Encontrar cadenas de caracteres que no debimos haber encontrado (dedo, después, adelante)
 - Errores por falsos positivos (tipo 1)
 - No encontrar cadenas de caracteres que debíamos de haber encontrado (De)
 - Errores por falsos negativos (tipo 2)
- En PLN tenemos que tratar constantemente estos dos tipos de errores. A menudo tenemos que hacerlo desde dos enfoques antagonistas
 - **Aumentar accuracy o precision** (minimizar falsos positivos)
 - **Aumentar recall** (minimizar falsos negativos)

- El proceso anterior se basó en tratar de arreglar dos tipos de errores
 - Encontrar cadenas de caracteres que no debimos haber encontrado (dedo, después, adelante)
 - Errores por falsos positivos (tipo 1)
 - No encontrar cadenas de caracteres que debíamos de haber encontrado (De)
 - Errores por falsos negativos (tipo 2)
- En PLN tenemos que tratar constantemente estos dos tipos de errores. A menudo tenemos que hacerlo desde dos enfoques antagonistas
 - Aumentar accuracy o precision (minimizar falsos positivos)
 - Aumentar recall (minimizar falsos negativos)

- El proceso anterior se basó en tratar de arreglar dos tipos de errores
 - Encontrar cadenas de caracteres que no debimos haber encontrado (dedo, después, adelante)
 - Errores por falsos positivos (tipo 1)
 - No encontrar cadenas de caracteres que debíamos de haber encontrado (De)
 - Errores por falsos negativos (tipo 2)
- En PLN tenemos que tratar constantemente estos dos tipos de errores. A menudo tenemos que hacerlo desde dos enfoques antagonistas
 - Aumentar accuracy o precision (minimizar falsos positivos)
 - Aumentar recall (minimizar falsos negativos)

- El proceso anterior se basó en tratar de arreglar dos tipos de errores
 - Encontrar cadenas de caracteres que no debimos haber encontrado (dedo, después, adelante)
 - Errores por falsos positivos (tipo 1)
 - No encontrar cadenas de caracteres que debíamos de haber encontrado (De)
 - Errores por falsos negativos (tipo 2)
- En PLN tenemos que tratar constantemente estos dos tipos de errores. A menudo tenemos que hacerlo desde dos enfoques antagonistas
 - **Aumentar accuracy o precision** (minimizar falsos positivos)
 - **Aumentar recall** (minimizar falsos negativos)

Búsqueda de expresiones regulares

Ejemplo de uso en python

```
In [1]: import re

mi_cadena = "Hagamos 1 prueba de Detección de 12 expresiones regulares"
PATRON = "\s+"
# Algunos Posibles patrones
# "[0-9]" "\s+" "\w+" "[a-z]" "\w" "[^a-z]" "[A-Z]" "a^g" "[^0-9]" "\." "\.$" ".$"
re.findall(PATRON, mi_cadena)
```

```
Out[1]: [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

```
In [2]: re.split(PATRON, mi_cadena)
```

```
Out[2]: ['Hagamos',
         '1',
         'prueba',
         'de',
         'Detección',
         'de',
         '12',
         'expresiones',
         'regulares']
```

```
In [3]: re.match('Hag', mi_cadena)
```

```
Out[3]: <re.Match object; span=(0, 3), match='Hag'>
```

```
In [4]: re.search('pru', mi_cadena)
```

```
Out[4]: <re.Match object; span=(10, 13), match='pru'>
```

Búsqueda de expresiones regulares

Ejemplo de uso en python

Para enlazar condiciones podemos usar el símbolo "|" (equivalente a OR)

```
In [24]: PATRON_digitos_palabras = ('(\d+|\w+)')
re.findall(PATRON_digitos_palabras, mi_cadena)
```

```
Out[24]: ['Hagamos',
          '1',
          'prueba',
          'de',
          'Detección',
          'de',
          '12',
          'expresiones',
          'regulares']
```

Para buscar rangos de cadenas de caracteres podemos poner condiciones anidadas

```
In [28]: mi_cadena = "esta cadena de caracteres tiene 12 números, pero no detecta las comas."
re.match('[0-9a-zAÉÍÓÚ ]+', mi_cadena)
```

```
Out[28]: <re.Match object; span=(0, 42), match='esta cadena de caracteres tiene 12 números'>
```


- En **PLN** y en **Lingüística computacional**, las expresiones regulares juegan un papel fundamental.
 - Estas técnicas de depuración de datos por medio de lenguajes formales son frecuentemente el primer modelo (etapa) de procesamiento de textos.
- En la mayoría de tareas, a partir de este punto empleamos técnicas de **aprendizaje de máquina**.
 - Por esta razón es importante ver este primer paso como una forma de extraer características limpias para nuestro sistema.
 - Puede llegar a ser muy útil en la captura de generalizaciones (más adelante veremos **lematización**).

- En **PLN** y en **Lingüística computacional**, las expresiones regulares juegan un papel fundamental.
 - Estas técnicas de depuración de datos por medio de lenguajes formales son frecuentemente el primer modelo (etapa) de procesamiento de textos.
- En la mayoría de tareas, a partir de este punto empleamos técnicas de **aprendizaje de máquina**.
 - Por esta razón es importante ver este primer paso como una forma de extraer características limpias para nuestro sistema.
 - Puede llegar a ser muy útil en la captura de generalizaciones (más adelante veremos **lematización**).

Contenido

- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization**
- 4 Procesamiento de texto - Normalización y lematización
- 5 Herramientas

- Toda tarea de PLN requiere de un paso de **normalización** de los textos de entrada
 - Segmentar/*tokenizar* palabras en el *running text*
 - Normalizar el formato de las palabras
 - Segmentar frases en el *running text*

- Toda tarea de PLN requiere de un paso de **normalización** de los textos de entrada
 - Segmentar/*tokenizar* palabras en el *running text*
 - Normalizar el formato de las palabras
 - Segmentar frases en el *running text*

- Toda tarea de PLN requiere de un paso de **normalización** de los textos de entrada
 - Segmentar/*tokenizar* palabras en el *running text*
 - Normalizar el formato de las palabras
 - Segmentar frases en el *running text*

¿Con cuántos elementos trabajamos?

La **tokenización** se puede hacer a nivel de frase o de palabra.
Veamos el caso de **tokenización** a nivel de palabra.

- **Yo ten- tengo ganas de salir hoy ehhe temprano**
 - Fragmentos, pausas, muletillas.
- El **algoritmo** genético es distinto a los **algoritmos** metaheurísticos
- **Lemma (raíz)**: la misma raíz, significado asociado
 - **algoritmo** y **algoritmos** = mismo lemma
- **Wordform (inflexión / flexión)**: Formas producto de flexiones gramaticales
 - **algoritmo** y **algoritmos** = distintas flexiones

¿Con cuántos elementos trabajamos?

La **tokenización** se puede hacer a nivel de frase o de palabra.

Veamos el caso de **tokenización** a nivel de palabra.

- **Yo ten- tengo ganas de salir hoy ehhe temprano**
 - Fragmentos, pausas, muletillas.
- El **algoritmo** genético es distinto a los **algoritmos** metaheurísticos
- **Lemma (raíz)**: la misma raíz, significado asociado
 - **algoritmo** y **algoritmos** = mismo lemma
- **Wordform (inflexión / flexión)**: Formas producto de flexiones gramaticales
 - **algoritmo** y **algoritmos** = distintas flexiones

¿Con cuántos elementos trabajamos?

La **tokenización** se puede hacer a nivel de frase o de palabra.

Veamos el caso de **tokenización** a nivel de palabra.

- **Yo ten- tengo ganas de salir hoy ehhh temprano**
 - Fragmentos, pausas, muletillas.
- El **algoritmo** genético es distinto a los **algoritmos** metaheurísticos
- **Lemma (raíz)**: la misma raíz, significado asociado
 - **algoritmo** y **algoritmos** = mismo lemma
- **Wordform (inflexión / flexión)**: Formas producto de flexiones gramaticales
 - **algoritmo** y **algoritmos** = distintas flexiones

¿Con cuántos elementos trabajamos?

ellos yacen de espalda en la playa de Cartagena mientras piensan en los que tienen que trabajar

- **Palabra:** un elemento del vocabulario
- **Token:** una instancia de cierta palabra en el *running text*
- ¿Cuántas tenemos?
 - Palabras: 14
 - Tokens: 17

¿Con cuántos elementos trabajamos?

- **N** = Número de tokens
- **V** = Vocabulario (conjunto de palabras); $|V|$ es el tamaño del vocabulario

	Tokens = N	Palabras = $ V $
Switchboard phone conversations	2.4 millón	20 mil
Shakespeare	884,000	31 mil
Google N-grams	1 trillón	13 millones

Simple Tokenization in UNIX

- (Inspired by Ken Church's UNIX for Poets.)
- Given a text file, output the word tokens and their frequencies

```
tr -sc 'A-Za-z' '\n' < shakes.txt
```

Change all non-alpha to newlines

```
| sort
```

Sort in alphabetical order

```
| uniq -c
```

Merge and count each type

```
1945 A          25 Aaron
  72 AARON      6 Abate
  19 ABBESS     1 Abates
   5 ABBOT      5 Abbess
... ..         6 Abbey
... ..         3 Abbot
... ..
```

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

```
THE  
SONNETS  
by  
William  
Shakespeare  
From  
fairest  
creatures  
We  
...
```

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

More counting

- Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

- Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
8954 d
```

What happened here?

Issues in Tokenization

- Finland's capital → Finland Finlands Finland's ?
- what're, I'm, isn't → What are, I am, is not
- Hewlett-Packard → Hewlett Packard ?
- state-of-the-art → state of the art ?
- Lowercase → lower-case lowercase lower case ?
- San Francisco → one token or two?
- m.p.h., PhD. → ??

Tokenization: language issues

- French
 - *L'ensemble* → one token or two?
 - *L ? L' ? Le ?*
 - Want *l'ensemble* to match with *un ensemble*
- German noun compounds are not segmented
 - *Lebensversicherungsgesellschaftsangestellter*
 - 'life insurance company employee'
 - German information retrieval needs **compound splitter**

Tokenization: language issues

- Chinese and Japanese no spaces between words:
 - 莎拉波娃现在居住在美国东南部的佛罗里达。
 - 莎拉波娃 现在 居住 在 美国 东南部 的 佛罗里达
 - Sharapova now lives in US southeastern Florida
- Further complicated in Japanese, with multiple alphabets intermingled
 - Dates/amounts in multiple formats

フォーチュン500社は情報不足のため時間あた\$500K(約6,000万円)

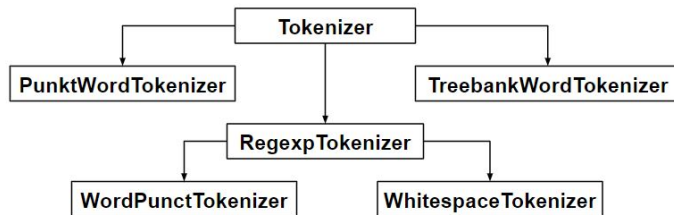
Katakana Hiragana Kanji Romaji

End-user can express query entirely in hiragana!

Ejemplos prácticos de tokenización

Vamos a ver inicialmente algunos ejemplos de tokenización empleando NLTK.
NLTK permite hacer tokenización a nivel de:

- Palabras (`word_tokenize`)
- Frases (`sent_tokenize`)
- Expresiones regulares (`regexp_tokenize`)
- Tweets (`TweetTokenizer`)



Ejemplos prácticos de tokenización

A nivel de palabra

```
In [7]: from nltk.tokenize import word_tokenize

text = "Hola a todos. Esta es nuestra primera tokenizada del curso."
word_tokenize(text)
```

```
Out[7]: ['Hola',
         'a',
         'todos',
         '.',
         'Esta',
         'es',
         'nuestra',
         'primera',
         'tokenizada',
         'del',
         'curso',
         '.']
```

```
In [8]: from nltk.tokenize import TreebankWordTokenizer

tokenizer = TreebankWordTokenizer()
tokenizer.tokenize(text)
```

```
Out[8]: ['Hola',
         'a',
         'todos.',
         'Esta',
         'es',
         'nuestra',
         'primera',
         'tokenizada',
         'del',
         'curso',
         '.']
```

Ejemplos prácticos de tokenización

A nivel de palabra

```
In [12]: from nltk.tokenize import WordPunctTokenizer

tokenizer = WordPunctTokenizer()
tokenizer.tokenize("Hola a todos. Let's see how NLTK's tokenizing.")
```

```
Out[12]: ['Hola',
          'a',
          'todos',
          '.',
          'Let',
          "'",
          's',
          'see',
          'how',
          'NLTK',
          "'",
          's',
          'tokenizing',
          '.']
```

```
In [14]: from nltk.tokenize import RegexpTokenizer

tokenizer = RegexpTokenizer("[\w']+")
tokenizer.tokenize(text)
```

```
Out[14]: ['Hola', 'a', 'todos', "Let's", 'see', 'how', "NLTK's", 'tokenizing']
```

Ejemplos prácticos de tokenización

A nivel de frase

```
In [16]: from nltk.tokenize import sent_tokenize

text = "Hola a todos. Esta es nuestra primera tokenizada del curso. ¡Vamos a ver qué tal sale!"
sent_tokenize(text)
```

```
Out[16]: ['Hola a todos.',
          'Esta es nuestra primera tokenizada del curso.',
          '¡Vamos a ver qué tal sale!']
```

```
In [14]: from nltk.tokenize import RegexpTokenizer

tokenizer = RegexpTokenizer("[\w']+")
tokenizer.tokenize(text)
```

```
Out[14]: ['Hola', 'a', 'todos', "Let's", 'see', 'how', "NLTK's", 'tokenizing']
```

Contenido

- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization
- 4 Procesamiento de texto - Normalización y lematización**
- 5 Herramientas

- Necesidad de normalizar textos
 - Desde el punto de vista de la **Recuperación de Información** (*Information Retrieval*), los términos que estén indexados deben ser iguales a los que usen para formular las búsquedas
 - Por ejemplo: Se desea que U.S.A. y USA sean procesados como si fueran los mismos términos
- Se pueden definir equivalencias implícitamente
 - e.g. eliminando los puntos en los términos, pero no siempre es la mejor solución (USA es una inflexión del verbo **usar**).
- Alternativa: realizar expansiones asimétricas
 - Query: window Búsqueda: window, windows
 - Query: windows Búsqueda: Windows, windows, window
 - Query: Windows Búsqueda: Windows
- Potencialmente, es **más poderoso**; sin embargo, es **menos eficiente** y **más difícil** de implementar.

- Necesidad de normalizar textos
 - Desde el punto de vista de la **Recuperación de Información** (*Information Retrieval*), los términos que estén indexados deben ser iguales a los que usen para formular las búsquedas
 - Por ejemplo: Se desea que U.S.A. y USA sean procesados como si fueran los mismos términos
- Se pueden definir equivalencias implícitamente
 - e.g. eliminando los puntos en los términos, pero no siempre es la mejor solución (USA es una inflexión del verbo **usar**).
- Alternativa: realizar expansiones asimétricas
 - Query: window Búsqueda: window, windows
 - Query: windows Búsqueda: Windows, windows, window
 - Query: Windows Búsqueda: Windows
- Potencialmente, es **más poderoso**; sin embargo, es **menos eficiente** y **más difícil** de implementar.

- Necesidad de normalizar textos
 - Desde el punto de vista de la **Recuperación de Información** (*Information Retrieval*), los términos que estén indexados deben ser iguales a los que usen para formular las búsquedas
 - Por ejemplo: Se desea que U.S.A. y USA sean procesados como si fueran los mismos términos
- Se pueden definir equivalencias implícitamente
 - e.g. eliminando los puntos en los términos, pero no siempre es la mejor solución (USA es una inflexión del verbo **usar**).
- Alternativa: realizar expansiones asimétricas
 - Query: window Búsqueda: window, windows
 - Query: windows Búsqueda: Windows, windows, window
 - Query: Windows Búsqueda: Windows
- Potencialmente, es **más poderoso**; sin embargo, es **menos eficiente** y **más difícil** de implementar.

- Necesidad de normalizar textos
 - Desde el punto de vista de la **Recuperación de Información** (*Information Retrieval*), los términos que estén indexados deben ser iguales a los que usen para formular las búsquedas
 - Por ejemplo: Se desea que U.S.A. y USA sean procesados como si fueran los mismos términos
- Se pueden definir equivalencias implícitamente
 - e.g. eliminando los puntos en los términos, pero no siempre es la mejor solución (USA es una inflexión del verbo **usar**).
- Alternativa: realizar expansiones asimétricas
 - Query: window Búsqueda: window, windows
 - Query: windows Búsqueda: Windows, windows, window
 - Query: Windows Búsqueda: Windows
- Potencialmente, es **más poderoso**; sin embargo, es **menos eficiente** y **más difícil** de implementar.

Dependiendo del campo de aplicación se pueden considerar ciertos aspectos

- El tratamiento de las mayúsculas es “irrelevante” desde el punto de vista de la IR
 - Los usuarios tienden a usar minúsculas
 - **chevrolet**, **utp**, **presidencia**, **ministerio**, **dian**
 - **Chevrolet**, **UTP**, **Presidencia**, **Ministerio**, **DIAN**
- Sin embargo, dentro del *Sentiment Analysis* (análisis de sentimientos, de opinión), o dentro del análisis de conversaciones (o comentarios), las mayúsculas juegan un papel importante.
 - “Te he dicho **QUE NO QUIERO SALIR CONTIGO**”
 - “La comida estaba sencillamente **DELICIOSA**”

Dependiendo del campo de aplicación se pueden considerar ciertos aspectos

- El tratamiento de las mayúsculas es “irrelevante” desde el punto de vista de la IR
 - Los usuarios tienden a usar minúsculas
 - **chevrolet**, **utp**, **presidencia**, **ministerio**, **dian**
 - **Chevrolet**, **UTP**, **Presidencia**, **Ministerio**, **DIAN**
- Sin embargo, dentro del *Sentiment Analysis* (análisis de sentimientos, de opinión), o dentro del análisis de conversaciones (o comentarios), las mayúsculas juegan un papel importante.
 - “Te he dicho **QUE NO QUIERO SALIR CONTIGO**”
 - “La comida estaba sencillamente **DELICIOSA**”

- Su objetivo es reducir inflexiones gramaticales propias del lenguaje
 - soy, eres, es, son → **ser**
 - estoy, están, estás, está → **estar**
 - carro, carrito, carrazo, carroza → **carro**
 - color, colores → **color**
- *el carrito del bebé es de distintos colores* → *el **carro** del bebé **ser** de distintos **color***
- La **lematización** necesita un diccionario del cual partir.

<http://nlp.cs.upc.edu/freeling/>

- Su objetivo es reducir inflexiones gramaticales propias del lenguaje
 - soy, eres, es, son → **ser**
 - estoy, están, estás, está → **estar**
 - carro, carrito, carrazo, carroza → **carro**
 - color, colores → **color**
- *el carrito del bebé es de distintos colores* → *el **carro** del bebé **ser** de distintos **color***
- La **lematización** necesita un diccionario del cual partir.

<http://nlp.cs.upc.edu/freeling/>

Ejemplo práctico de lematización

Con WordNet

```
In [2]: #import nltk
#nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

print("piedras :", lemmatizer.lemmatize("piedras"))
print("carros :", lemmatizer.lemmatize("carros"))

print("books :", lemmatizer.lemmatize("books"))
print("classes :", lemmatizer.lemmatize("classes"))

# a denotes adjective in "pos"
print("better :", lemmatizer.lemmatize("better", pos = "a"))
print("harder :", lemmatizer.lemmatize("harder", pos = "a"))
```

```
piedras : piedras
carros : carros
books : book
classes : class
better : good
harder : hard
```

```
In [3]: # Create WordNetLemmatizer object
wnl = WordNetLemmatizer()

# single word lemmatization examples
list1 = ['kites', 'babies', 'dogs', 'flying', 'smiling',
         'driving', 'died', 'tried', 'feet']
for words in list1:
    print(words + " --> " + wnl.lemmatize(words))
```

```
kites --> kite
babies --> baby
dogs --> dog
flying --> flying
smiling --> smiling
driving --> driving
died --> died
tried --> tried
feet --> foot
```


Ejemplo práctico de lematización

Con spacy

```
In [4]: import spacy
nlp = spacy.load('en_core_web_sm')

# Create a Doc object
doc = nlp(u'the bats saw the cats with best stripes hanging upside down by their feet')

# Create List of tokens from given string
tokens = []
for token in doc:
    tokens.append(token)

print(tokens)
#> [the, bats, saw, the, cats, with, best, stripes, hanging, upside, down, by, their, feet]

lemmatized_sentence = " ".join([token.lemma_ for token in doc])

print(lemmatized_sentence)
#> the bat see the cat with good stripe hang upside down by -PRON- foot

[the, bats, saw, the, cats, with, best, stripes, hanging, upside, down, by, their, feet]
the bat see the cat with good stripe hang upside down by -PRON- foot
```

- Sigue el mismo objetivo de la lematización; sin embargo, éste reduce los términos a sus raíces gramaticales
- Puede verse como un **corte en crudo** de los sufijos
 - Depende del lenguaje (de ahí que necesite un diccionario)
 - e.g. *pereza*, *perezoso*, *perezosa* se reducen a *perez*
- **Tarea:** Investigar el algoritmo de Porter para la **lematización** y **stemming** en inglés.

- Sigue el mismo objetivo de la lematización; sin embargo, éste reduce los términos a sus raíces gramaticales
- Puede verse como un **corte en crudo** de los sufijos
 - Depende del lenguaje (de ahí que necesite un diccionario)
 - e.g. *pereza*, *perezoso*, *perezosa* se reducen a *perez*
- **Tarea:** Investigar el algoritmo de Porter para la lematización y stemming en inglés.

- Sigue el mismo objetivo de la lematización; sin embargo, éste reduce los términos a sus raíces gramaticales
- Puede verse como un **corte en crudo** de los sufijos
 - Depende del lenguaje (de ahí que necesite un diccionario)
 - e.g. *pereza*, *perezoso*, *perezosa* se reducen a *perez*
- **Tarea:** Investigar el algoritmo de Porter para la **lematización** y **stemming** en inglés.

- Algunos idiomas requieren una segmentación más compleja a nivel de **morfemas**
 - En idioma **turco**
 - **Uygarlastiramadiklarimizdanmissinizcasina**
 - “Compórtate como si fueras de aquellos a quienes no pudimos civilizar”
 - **Uygar** “civilizar” + **las** “llegar a ser” + **tir** “causa” + **ama** “no poder” + **dik** “pasado” + **lar** “plural” + ...

Contenido

- 1 Introducción
- 2 Procesamiento de texto - Expresiones regulares
- 3 Procesamiento de texto - Tokenization
- 4 Procesamiento de texto - Normalización y lematización
- 5 Herramientas**

- <https://opensource.com/business/15/7/five-open-source-nlp-tools>
 - <http://www.programcreek.com/2013/05/collection-of-natural-language-processing-tools/>
 - <http://www.phontron.com/nlptools.php>
 - <http://molinodeideas.com/index.html>
 - <http://nlp.stanford.edu/software/>
-
- <https://www ldc.upenn.edu/>
 - <http://childes.psy.cmu.edu/>
 - <http://www.natcorp.ox.ac.uk/>
 - <http://www.gutenberg.org/>

- <https://opensource.com/business/15/7/five-open-source-nlp-tools>
- <http://www.programcreek.com/2013/05/collection-of-natural-language-processing-tools/>
- <http://www.phontron.com/nlptools.php>
- <http://molinodeideas.com/index.html>
- <http://nlp.stanford.edu/software/>

- <https://www ldc.upenn.edu/>
- <http://childes.psy.cmu.edu/>
- <http://www.natcorp.ox.ac.uk/>
- <http://www.gutenberg.org/>